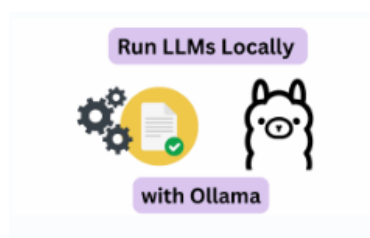


Ollama 教程



Ollama 是一个开源的本地大语言模型运行框架，专为在本地机器上便捷部署和运行大型语言模型（LLM）而设计。

Ollama 支持多种操作系统，包括 macOS、Windows、Linux 以及通过 Docker 容器运行。

Ollama 提供对模型量化的支持，可以显著降低显存要求，使得在普通家用计算机上运行大型模型成为可能。

谁适合阅读本教程？

Ollama 适用于开发者、研究人员以及对数据隐私有较高要求的用户，它可以帮助用户在本地环境中快速部署和运行大型语言模型，同时提供灵活的定制化选项。

使用 Ollama，我们可以在本地运行 Llama 3.3、DeepSeek-R1、Phi-4、Mistral、Gemma 2 和其他模型。

学习本教程前你需要了解

本教程适合有 Python 基础的开发者学习，如果不了解 Python 可以查阅 [Python 3.x 基础教程](#)。理解 Docker 镜像和容器的区别，知道如何从 Docker Hub 拉取镜像并运行容器，docker 相关内容参见： [Docker 教程](#)。

熟悉命令行工具（如终端或命令提示符）的基本操作，例如文件和目录的创建、删除、移动，以及如何运行脚本和程序。

创建新的模型

我们可以使用 `ollama create` 命令从 Modelfile 创建模型：

实例

```
ollama create model -of ./Modelfile
```

相关链接

Ollama 官方地址: <https://ollama.com/>

Github 开源地址: <https://github.com/ollama/ollama>

Ollama 官方文档: <https://github.com/ollama/ollama/tree/main/docs>

Ollama 简介

Ollama 是一个开源的大型语言模型（LLM）平台，旨在让用户能够轻松地在本地图运行、管理和与大型语言模型进行交互。

Ollama 提供了一个简单的方式来加载和使用各种预训练的语言模型，支持文本生成、翻译、代码编写、问答等多种自然语言处理任务。

Ollama 的特点在于它不仅提供了现成的模型和工具集，还提供了方便的界面和 API，使得从文本生成、对话系统到语义分析等任务都能快速实现。

与其他 NLP 框架不同，Ollama 旨在简化用户的工作流程，使得机器学习不再是只有深度技术背景的开发才能触及的领域。

Ollama 支持多种硬件加速选项，包括纯 CPU 推理和各类底层计算架构（如 Apple Silicon），能够更好地利用不同类型的硬件资源。

核心功能与特点

1. 多种预训练语言模型支持

Ollama 提供了多种开箱即用的预训练模型，包括常见的 GPT、BERT 等大型语言模型。用户可以轻松加载并使用这些模型进行文本生成、情感分析、问答等任务。

2. 易于集成和使用

Ollama 提供了命令行工具（CLI）和 Python SDK，简化了与其他项目和服务的集成。开发者无需担心复杂的依赖或配置，可以快速将 Ollama 集成到现有的应用中。

3. 本地部署与离线使用

不同于一些基于云的 NLP 服务，Ollama 允许开发者在本地计算环境中运行模型。这意味着可以脱离对外部服务器的依赖，保证数据隐私，并且对于高并发的请求，离线部署能提供更低的延迟和更高的可控性。

4. 支持模型微调与自定义

用户不仅可以使⤵ Ollama 提供的预训练模型，还可以在此基础上进行模型微调。根据自

己的特定需求，开发者可以使用自己收集的数据对模型进行再训练，从而优化模型的性能和准确度。

5. 性能优化

Ollama 关注性能，提供了高效的推理机制，支持批量处理，能够有效管理内存和计算资源。这让它在处理大规模数据时依然保持高效。

6. 跨平台支持

Ollama 支持在多个操作系统上运行，包括 Windows、macOS 和 Linux。这样无论是开发者在本地环境调试，还是企业在生产环境部署，都能得到一致的体验。

7. 开放源码与社区支持

Ollama 是一个开源项目，这意味着开发者可以查看源代码，进行修改和优化，也可以参与到项目的贡献中。此外，Ollama 有一个活跃的社区，开发者可以从中获取帮助并与其他人交流经验。

应用场景

1. 内容创作：

帮助作家、记者、营销人员快速生成高质量的内容，例如博客文章、广告文案等。

2. 编程辅助：

帮助开发者生成代码、调试程序或优化代码结构。

3. 教育和研究：

辅助学生和研究人员进行学习、写作和研究，例如生成论文摘要或解答问题。

4. 跨语言交流：

提供高质量的翻译功能，帮助用户打破语言障碍。

5. 个人助手：

作为一个智能助手，帮助用户完成日常任务，例如撰写邮件、生成待办事项等。

Ollama 与其他 LLM 的区别

区别维度	Ollama 的特点	说明
本地化	更注重本地运行	与 ChatGPT 等依赖云服务的 LLM 不同，适合对数据隐私要求较高的用户
灵活性	可加载不同模型	用户可以根据需要加载不同的模型，而无需局限于单一的模型
开源	开源项目	用户可以自由地修改和扩展其功能

Ollama 安装

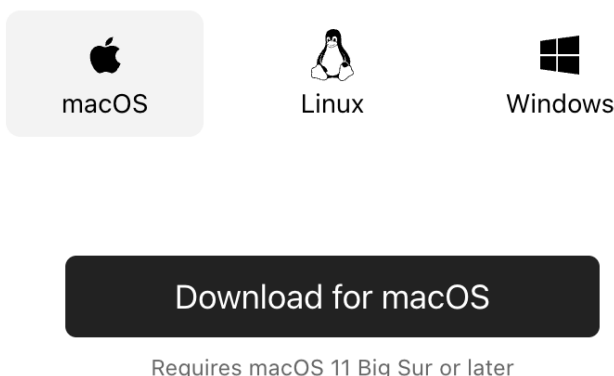
Ollama 支持多种操作系统，包括 macOS、Windows、Linux 以及通过 Docker 容器运行。

Ollama 对硬件要求不高，旨在让用户能够轻松地在本机运行、管理和与大型语言模型进行交互。

- **CPU:** 多核处理器（推荐 4 核或以上）。
- **GPU:** 如果你计划运行大型模型或进行微调，推荐使用具有较高计算能力的 GPU（如 NVIDIA 的 CUDA 支持）。
- **内存:** 至少 8GB RAM，运行较大模型时推荐 16GB 或更高。
- **存储:** 需要足够的硬盘空间来存储预训练模型，通常需要 10GB 至数百 GB 的空间，具体取决于模型的大小。
- **软件要求:** 确保系统上安装了最新版本的 Python（如果打算使用 Python SDK）。

Ollama 官方下载地址: <https://ollama.com/download>。

Download Ollama



我们可以根据不同的系统下载对应的包。

1、Windows 系统安装

打开浏览器，访问 Ollama 官方网站: <https://ollama.com/download>，下载适用于 Windows 的安装程序。

下载地址为: <https://ollama.com/download/OllamaSetup.exe>。

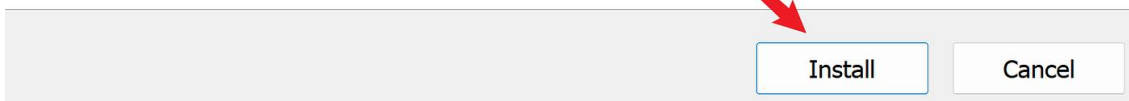
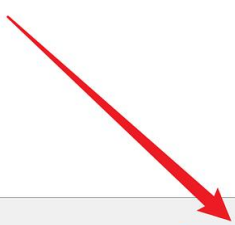
下载完成后，双击安装程序并按照提示完成安装。

Ollama Windows Preview



Let's get you up and running with your own large language models.

Click Install to continue with the installation.

**Install****Cancel**

验证安装

打开命令提示符或 PowerShell，输入以下命令验证安装是否成功：

```
ollama --version
```

如果显示版本号，则说明安装成功。

更改安装路径（可选）

如果需要将 Ollama 安装到非默认路径，可以在安装时通过命令行指定路径，例如：

```
OllamaSetup.exe /DIR="d:\some\location"
```

这样可以将 Ollama 安装到指定的目录。

2、macOS 系统安装

打开浏览器，访问 Ollama 官方网站：<https://ollama.com/download>，下载适用于 macOS 的安装程序。

下载地址为: <https://ollama.com/download/Ollama-darwin.zip>。

下载完成后, 双击安装包并按照提示完成安装。

安装完成后, 通过以下命令验证:

```
ollama --version
```

如果显示版本号, 则说明安装成功。

3、Linux 系统安装

Linux 下可以使用一键安装脚本, 我们打开终端, 运行以下命令:

```
curl -fsSL https://ollama.com/install.sh | bash
```

安装完成后, 通过以下命令验证:

```
ollama --version
```

如果显示版本号, 则说明安装成功。

4、Docker 安装

如果你熟悉 Docker, 也可以通过 Docker 安装 Ollama。

官方 Docker 镜像 ollama/ollama 可在 Docker Hub 上获取: <https://hub.docker.com/r/ollama/ollama>。

拉取 Docker 镜像:

```
docker pull ollama/ollama
```

运行容器:

```
docker run -p 11434:11434 ollama/ollama
```

访问 <http://localhost:11434> 即可使用 Ollama。

Ollama 运行模型

Ollama 运行模型使用 `ollama run` 命令。

例如我们要运行 Llama 3.2 并与该模型对话可以使用以下命令：

```
ollama run llama3.2
```

执行以上命令如果没有该模型会去下载 llama3.2 模型：

```
pulling manifest
pulling dde5aa3fc5ff... 80% ██████████ | 1.6 GB/2.0 GB 327 KB/s
```

等待下载完成后，我们在终端中，输入以下命令来加载 LLama3.2 模型并进行交互：

```
writing manifest
success
>>> 你好
Hello

>>> 能讲中文吗
是的，我可以在 Chinese 中对话。哪些话题或问题想要了解我呢？
```

结束对话可以输入 `/bye` 或按 `Ctrl+d` 按键来结束。

我们可以使用 `ollama list`，查看已安装的模型：

NAME	ID	SIZE	MODIFIED
llama3.2	baf6a787fdff	1.3 GB	4 minutes ago

Ollama 支持的模型可以访问：<https://ollama.com/library>



Library

Popular ▾

deepseek-r1

DeepSeek's first-generation of reasoning models with comparable performance to OpenAI-o1, including six dense models distilled from DeepSeek-R1 based on Llama and Qwen.

1.5b 7b 8b 14b 32b 70b 671b

↓ 8.9M Pulls 🏷 28 Tags 🕒 Updated 2 weeks ago

下表列出一些模型的下载命令：

模型	参数	大小	下载命令
Llama 3.3	70B	43GB	ollama run llama3.3

模型	参数	大小	下载命令
Llama 3.2	3B	2.0GB	ollama run llama3.2
Llama 3.2	1B	1.3GB	ollama run llama3.2:1b
Llama 3.2 Vision	11B	7.9GB	ollama run llama3.2-vision
Llama 3.2 Vision	90B	55GB	ollama run llama3.2-vision:90b
Llama 3.1	8B	4.7GB	ollama run llama3.1
Llama 3.1	405B	231GB	ollama run llama3.1:405b
Phi 4	14B	9.1GB	ollama run phi4
Phi 3 Mini	3.8B	2.3GB	ollama run phi3
Gemma 2	2B	1.6GB	ollama run gemma2:2b
Gemma 2	9B	5.5GB	ollama run gemma2
Gemma 2	27B	16GB	ollama run gemma2:27b
Mistral	7B	4.1GB	ollama run mistral
Moondream 2	1.4B	829MB	ollama run moondream
Neural Chat	7B	4.1GB	ollama run neural-chat
Starling	7B	4.1GB	ollama run starling-lm
Code Llama	7B	3.8GB	ollama run codellama
Llama 2 Uncensored	7B	3.8GB	ollama run llama2-uncensored

模型	参数	大小	下载命令
LLaVA	7B	4.5GB	ollama run llava
Solar	10.7B	6.1GB	ollama run solar

通过 Python SDK 使用模型

如果你希望将 Ollama 与 Python 代码集成，可以使用 Ollama 的 Python SDK 来加载和运行模型。

1. 安装 Python SDK

首先，需要安装 Ollama 的 Python SDK，打开终端，执行以下命令：

```
pip install ollama
```

2. 编写 Python 脚本

接下来，你可以使用 Python 代码来加载和与模型交互。

以下是一个简单的 Python 脚本示例，演示如何使用 LLama3.2 模型来生成文本：

实例

```
import ollama
response = ollama.generate(
    model="llama3.2", # 模型名称
    prompt="你是谁。" # 提示文本
)
print(response)
```

3. 运行 Python 脚本

在终端中运行你的 Python 脚本：

```
python test.py
```

你会看到模型根据你的输入返回的回答。

4. 对话模式

实例

```
from ollama import chat
response = chat(
    model="llama3.2",
    messages=[
        {"role": "user", "content": "为什么天空是蓝色的?" }
    ]
)
print(response.message.content)
```

此代码会与模型进行对话，并打印模型的回复。

5. 流式响应

实例

```
from ollama import chat
stream = chat(
    model="llama3.2",
    messages=[{"role": "user", "content": "为什么天空是蓝色的?" }],
    stream=True
)
for chunk in stream:
    print(chunk["message"]["content"], end="", flush=True)
```

此代码会以流式方式接收模型的响应，适用于处理大数据。

Ollama 相关命令

Ollama 提供了多种命令行工具（CLI）供用户与本地运行的模型进行交互。

我们可以用 `ollama --help` 查看包含有哪些命令：

Large language model runner

Usage:

```
ollama [flags]
ollama [command]
```

Available Commands:

serve	Start ollama
create	Create a model from a Modelfile
show	Show information for a model
run	Run a model
stop	Stop a running model
pull	Pull a model from a registry
push	Push a model to a registry
list	List models
ps	List running models
cp	Copy a model
rm	Remove a model
help	Help about any command

Flags:

-h, --help	help for ollama
-v, --version	Show version information

1、使用方法

- `ollama [flags]`: 使用标志（flags）运行 ollama。
- `ollama [command]`: 运行 ollama 的某个具体命令。

2、可用命令

- `serve`: 启动 ollama 服务。
- `create`: 根据一个 Modelfile 创建一个模型。
- `show`: 显示某个模型的详细信息。
- `run`: 运行一个模型。

- **stop**: 停止一个正在运行的模型。
- **pull**: 从一个模型仓库（**registry**）拉取一个模型。
- **push**: 将一个模型推送到一个模型仓库。
- **list**: 列出所有模型。
- **ps**: 列出所有正在运行的模型。
- **cp**: 复制一个模型。
- **rm**: 删除一个模型。
- **help**: 获取关于任何命令的帮助信息。

3、标志（Flags）

- **-h, --help**: 显示 **ollama** 的帮助信息。
- **-v, --version**: 显示版本信息。

1. 模型管理

拉取模型

从模型库中下载模型：

```
ollama pull <model-name>
```

例如：

```
ollama pull llama2
```

运行模型

运行已下载模型：

```
ollama run <model-name>
```

例如：

```
ollama run llama2
```

列出本地模型

查看已下载模型列表：

```
ollama list
```

删除模型

删除本地模型：

```
ollama rm <model-name>
```

例如：

```
ollama rm llama2
```

2. 自定义模型

创建自定义模型

基于现有模型创建自定义模型：

```
ollama create <custom-model-name> -f <Modelfile>
```

例如：

```
ollama create my-llama2 -f ./Modelfile
```

复制模型

复制一个已存在的模型：

```
ollama cp <source-model-name> <new-model-name>
```

例如：

```
ollama cp llama2 my-llama2-copy
```

推送自定义模型

将自定义模型推送到模型库：

```
ollama push <model-name>
```

例如：

```
ollama push my-llama2
```

3. 服务管理

启动 Ollama 服务

启动 Ollama 服务以在后台运行：

```
ollama serve
```

停止 Ollama 服务

停止正在运行的 Ollama 服务：

```
ollama stop
```

重启 Ollama 服务

重启 Ollama 服务：

```
ollama restart
```

4. 其他常用命令

查看帮助

查看所有可用命令：

```
ollama --help
```

查看版本信息

查看当前安装的 Ollama 版本：

```
ollama version
```

更新 Ollama

更新 Ollama 到最新版本：

```
ollama update
```

查看日志

查看 Ollama 的日志信息：


```
ollama logs
```

清理缓存

清理 Ollama 的缓存：

```
ollama clean
```

5. 模型信息

查看模型详细信息

查看指定模型的详细信息：

```
ollama show <model-name>
```

例如：

```
ollama show llama2
```

查看模型依赖

查看模型的依赖关系：

```
ollama deps <model-name>
```

例如：

```
ollama deps llama2
```

查看模型配置

查看模型的配置文件：

```
ollama config <model-name>
```

例如：

```
ollama config llama2
```

6. 导入与导出

导出模型

将模型导出为文件：

```
ollama export <model-name> <output-file>
```

例如：

```
ollama export llama2 llama2.tar
```

导入模型

从文件导入模型：

```
ollama import <input-file>
```

例如：

```
ollama import llama2.tar
```

7. 系统信息

查看系统信息

查看 Ollama 的系统信息：

```
ollama system
```

查看资源使用情况

查看模型的资源使用情况：

```
ollama resources <model-name>
```

例如：

```
ollama resources llama2
```

8. 模型性能

查看模型性能

查看模型的性能指标：

```
ollama perf <model-name>
```

例如：

```
ollama perf llama2
```

9. 模型历史

查看模型历史记录

查看模型的历史记录：

```
ollama history <model-name>
```

例如：

```
ollama history llama2
```

10. 模型状态

检查模型状态

检查指定模型的状态：

```
ollama status <model-name>
```

例如：

```
ollama status llama2
```

Ollama 基本概念

Ollama 是一个本地化的、支持多种自然语言处理 (NLP) 任务的机器学习框架，专注于模型加载、推理和生成任务。

通过 Ollama，用户能够方便地与本地部署的大型预训练模型进行交互。

1. 模型 (Model)

在 Ollama 中，模型是核心组成部分。它们是经过预训练的机器学习模型，能够执行不同的任务，例如文本生成、文本摘要、情感分析、对话生成等。

Ollama 支持多种流行的预训练模型，常见的模型有：

- **deepseek-v3**: 深度求索提供的大型语言模型，专门用于文本生成任务。
- **LLama2**: Meta 提供的大型语言模型，专门用于文本生成任务。
- **GPT**: OpenAI 的 GPT 系列模型，适用于广泛的对话生成、文本推理等任务。
- **BERT**: 用于句子理解和问答系统的预训练模型。
- **其他自定义模型**: 用户可以上传自己的自定义模型，并利用 Ollama 进行推理。

模型的主要功能：

- **推理 (Inference)**: 根据用户输入生成输出结果。
- **微调 (Fine-tuning)**: 用户可以在已有模型的基础上使用自己的数据进行训练，从而定制化模型以适应特定的任务或领域。

模型通常是由大量参数构成的神经网络，通过对大量文本数据进行训练，能够学习语言规律并进行高效的推理。

Ollama 支持的模型可以访问：<https://ollama.com/library>



Library

Filter models

Popular



deepseek-r1

DeepSeek's first-generation of reasoning models with comparable performance to OpenAI-o1, including six dense models distilled from DeepSeek-R1 based on Llama and Qwen.

1.5b 7b 8b 14b 32b 70b 671b

↓ 8.9M Pulls 🏷 28 Tags ⌚ Updated 2 weeks ago

点击模型，可以查看到下载的命令：

deepseek-r1

DeepSeek's first-generation of reasoning models with comparable performance to OpenAI-o1, including six dense models distilled from DeepSeek-R1 based on Llama and Qwen.

1.5b 7b 8b 14b 32b 70b 671b

↓ 8.9M Pulls ⌚ Updated 2 weeks ago

点击复制命令

7b



🏷 28 Tags

ollama run deepseek-r1



Updated 2 weeks ago

0a8c26691023 · 4.7GB

model arch **qwen2** · parameters **7.62B** · quantization **Q4_K_M** 4.7GB

params { "stop": ["<|begin_of_sentence|>", "<|end_of_sentence..." 148B

template {{- if .System }}{{ .System }}{{ end }} {{- range \$i, \$_ :... 387B

license MIT License Copyright (c) 2023 DeepSeek Permission is here... 1.1kB

下表列出一些模型的下载命令：

模型	参数	大小	下载命令
Llama 3.3	70B	43GB	ollama run llama3.3

模型	参数	大小	下载命令
Llama 3.2	3B	2.0GB	ollama run llama3.2
Llama 3.2	1B	1.3GB	ollama run llama3.2:1b
Llama 3.2 Vision	11B	7.9GB	ollama run llama3.2-vision
Llama 3.2 Vision	90B	55GB	ollama run llama3.2-vision:90b
Llama 3.1	8B	4.7GB	ollama run llama3.1
Llama 3.1	405B	231GB	ollama run llama3.1:405b
Phi 4	14B	9.1GB	ollama run phi4
Phi 3 Mini	3.8B	2.3GB	ollama run phi3
Gemma 2	2B	1.6GB	ollama run gemma2:2b
Gemma 2	9B	5.5GB	ollama run gemma2
Gemma 2	27B	16GB	ollama run gemma2:27b
Mistral	7B	4.1GB	ollama run mistral
Moondream 2	1.4B	829MB	ollama run moondream
Neural Chat	7B	4.1GB	ollama run neural-chat
Starling	7B	4.1GB	ollama run starling-lm
Code Llama	7B	3.8GB	ollama run codellama
Llama 2 Uncensored	7B	3.8GB	ollama run llama2-uncensored

模型	参数	大小	下载命令
LLaVA	7B	4.5GB	ollama run llava
Solar	10.7B	6.1GB	ollama run solar

2. 任务（Task）

Ollama 支持多种 NLP 任务。每个任务对应模型的不同应用场景，主要包括但不限于以下几种：

- **对话生成（Chat Generation）**：通过与用户交互生成自然的对话回复。
- **文本生成（Text Generation）**：根据给定的提示生成自然语言文本，例如写文章、生成故事等。
- **情感分析（Sentiment Analysis）**：分析给定文本的情感倾向（如正面、负面、中立）。
- **文本摘要（Text Summarization）**：将长文本压缩为简洁的摘要。
- **翻译（Translation）**：将文本从一种语言翻译成另一种语言。

通过命令行工具，用户可以指定不同的任务，并加载不同的模型来完成特定任务。

3. 推理（Inference）

推理是指在已训练的模型上进行输入处理，生成输出的过程。

Ollama 提供了易于使用的命令行工具或 API，使用户可以快速向模型提供输入并获取结果。

推理是 Ollama 的主要功能之一，也是与模型交互的核心。

推理过程：

1. **输入**：用户向模型提供文本输入，可以是一个问题、提示或者对话内容。
2. **模型处理**：模型通过内置的神经网络根据输入生成适当的输出。
3. **输出**：模型返回生成的文本内容，可能是回复、生成的文章、翻译文本等。

Ollama 通过 API 或 CLI 与本地模型交互，能够让用户轻松实现推理任务。

4. 微调（Fine-tuning）

微调是指在一个已预训练的模型上，基于特定的领域数据进行进一步的训练，以便使模型在特定任务或领域上表现得更好。

Ollama 支持微调功能，用户可以使用自己的数据集对预训练模型进行微调，来定制模型的输出。

微调过程：

-
1. **准备数据集：**用户准备特定领域的数据集，数据格式通常为文本文件或 JSON 格式。
 2. **加载预训练模型：**选择一个适合微调的预训练模型，例如 LLama2 或 GPT 模型。
 3. **训练：**使用用户的特定数据集对模型进行训练，使其能够更好地适应目标任务。
 4. **保存和部署：**训练完成后，微调过的模型可以保存并部署，供以后使用。

微调有助于模型在处理特定领域问题时表现得更加精确和高效。

Ollama 模型交互

Ollama 提供了多种方式与模型进行交互，其中最常见就是通过命令行进行推理操作。

1. 命令行交互

通过命令行直接与模型进行交互是最简单的方式。

运行模型

使用 `ollama run` 命令启动模型并进入交互模式：

```
ollama run <model-name>
```

例如我们下载 `deepseek-coder` 模型：

实例

```
ollama run deepseek-coder
```

启动后，您可以直接输入问题或指令，模型会实时生成响应。

```
>>> 你好，你能帮我写一段代码吗？
```

当然可以。但是首先我想知道您希望在哪种编程语言中实现这个功能（例如 `Python`、`JavaScript` 等）和要解决什么问题或者完成的任务是什么样的例子呢？这样我们可以为您提供更准确的内容，同时也方便我帮助你写出最适合您的代码片段。

```
>>> 写一段 python hello world
```

当然可以！这是一个简单的 `"Hello, World!"` 程序：

```
```python
print("Hello, World!")
```
```

这个脚本会输出 ``Hello, World!``，并将其打印到控制台上。这只是最基本的 `Python Hello world` 示例：`Python` 是一种解释型、通用型的编程语言以其简洁性和易读性而闻名。它还允许用户在代码中插入变量和表达式来创建复杂的行为。

退出交互模式

在交互模式下，输入 `/bye` 或按下 `Ctrl+d` 退出。

2. 单次命令交互

如果您只需要模型生成一次响应，可以直接在命令行中传递输入。

使用管道输入

通过管道将输入传递给模型：

实例

```
echo "你是谁?" | ollama run deepseek-coder
```

输出结果如下：

我是由中国的深度求索（DeepSeek）公司开发的编程智能助手，名为 DeepCoder。我专注于解答计算机科学相关的问题和任务。如果你有任何关于这个领域的话题或者需要帮助的地方，请随时提问！

使用命令行参数

直接在命令行中传递输入：

```
ollama run deepseek-coder "Python 的 hello world 代码?"
```

输出结果如下：

在 Python 中，"Hello World!" 通常是这段简单的脚本：

```
```python
print("Hello World!")
```
```

当你运行这个程序时，它会输出 `Hello, World`。这是因为 `print()` 函数将字符串 "Hello, World" 打印到标准输出设备（`stdout`） - 也就是你的屏幕上显示的信息（在这种情况下是命令行终端或类似的工具中运行 Python 脚本时，它会直接写入控制台。

3. 多轮对话

Ollama 支持多轮对话，模型可以记住上下文。

实例

```
>>> 你好，你能帮我写一段 Python 代码吗？
```

当然可以！请告诉我你需要实现什么功能。

```
>>> 我想写一个计算斐波那契数列的函数。
```

好的，以下是一个简单的 Python 函数：

```
def fibonacci(n):
    if n <= 1:
        return n
    else:
        return fibonacci(n-1) + fibonacci(n-2)
```

4. 文件输入

可以将文件内容作为输入传递给模型。

假设 input.txt 文件内容为：

```
Python 的 hello world 代码？
```

将 input.txt 文件内容作为输入：

```
ollama run deepseek-coder < input.txt
```

5. 自定义提示词

通过 Modelfile 定义自定义提示词或系统指令，使模型在交互中遵循特定规则。

创建自定义模型

编写一个 Modelfile：

实例

```
FROM deepseek-coder
```

```
SYSTEM "你是一个编程助手，专门帮助用户编写代码。"
```

然后创建自定义模型：

```
ollama create runoob-coder -f ./Modelfile
```

运行自定义模型：

```
ollama run runoob-coder
```

6. 交互日志

Ollama 会记录交互日志，方便调试和分析。

查看日志：

```
ollama logs
```

Ollama API 交互

Ollama 提供了基于 HTTP 的 API，允许开发者通过编程方式与模型进行交互。

本文将详细介绍 Ollama API 的详细使用方法，包括请求格式、响应格式以及示例代码。

1. 启动 Ollama 服务

在使用 API 之前，需要确保 Ollama 服务正在运行。可以通过以下命令启动服务：

```
ollama serve
```

默认情况下，服务会运行在 `http://localhost:11434`。

2. API 端点

Ollama 提供了以下主要 API 端点：

生成文本（Generate Text）

- 端点: POST /api/generate
- 功能: 向模型发送提示词（prompt），并获取生成的文本。
- 请求格式:

```
{
  "model": "<model-name>", // 模型名称
  "prompt": "<input-text>", // 输入的提示词
  "stream": false,         // 是否启用流式响应（默认 false）
  "options": {              // 可选参数
    "temperature": 0.7,     // 温度参数
    "max_tokens": 100       // 最大 token 数
  }
}
```

- 响应格式:

```
{
  "response": "<generated-text>", // 生成的文本
  "done": true                    // 是否完成
}
```

聊天（Chat）

- 端点: POST /api/chat
- 功能: 支持多轮对话，模型会记住上下文。

- 请求格式:

```
• {
•   "model": "<model-name>", // 模型名称
•   "messages": [           // 消息列表
•     {
•       "role": "user",      // 用户角色
•       "content": "<input-text>" // 用户输入
•     }
•   ],
•   "stream": false,        // 是否启用流式响应
•   "options": {            // 可选参数
•     "temperature": 0.7,
•     "max_tokens": 100
•   }
• }
```

- 响应格式:

```
• {
•   "message": {
•     "role": "assistant",  // 助手角色
•     "content": "<generated-text>" // 生成的文本
•   },
•   "done": true
• }
```

列出本地模型（List Models）

- 端点: GET /api/tags

- 功能: 列出本地已下载的模型。

- 响应格式:

```
• {
•   "models": [
•     {
•       "name": "<model-name>", // 模型名称
•       "size": "<model-size>", // 模型大小
•       "modified_at": "<timestamp>" // 修改时间
•     }
•   ]
• }
```

拉取模型（Pull Model）

- 端点: POST /api/pull
- 功能: 从模型库中拉取模型。
- 请求格式:
 - {
 - "name": "<model-name>" // 模型名称
 - }
- 响应格式:
 - {
 - "status": "downloading", // 下载状态
 - "digest": "<model-digest>" // 模型摘要
 - }

3. 使用示例

生成文本

使用 curl 发送请求:

实例

```
curl http://localhost:11434/api/generate -d '{
  "model": "deepseek-coder",
  "prompt": "你好，你能帮我写一段代码吗？",
  "stream": false
}'
```

多轮对话

使用 curl 发送请求:

实例

```
curl http://localhost:11434/api/chat -d '{
  "model": "deepseek-coder",
  "messages": [
    {
      "role": "user",
      "content": "你好，你能帮我写一段 Python 代码吗？"
```

```
    }  
  ],  
  "stream": false  
}'
```

列出本地模型

使用 curl 发送请求：

```
curl http://localhost:11434/api/tags
```

拉取模型

使用 curl 发送请求：

实例

```
curl http://localhost:11434/api/pull -d '{  
  "name": "deepseek-coder"  
}'
```

4. 流式响应

Ollama 支持流式响应（streaming response），适用于实时生成文本的场景。

启用流式响应

在请求中设置 **"stream": true**，API 会逐行返回生成的文本。

实例

```
curl http://localhost:11434/api/generate -d '{  
  "model": "deepseek-coder",  
  "prompt": "你好，你能帮我写一段代码吗？",  
  "stream": true  
}'
```

响应格式

每行返回一个 JSON 对象：

实例

```
{
```

```
"response": "<partial-text>", // 部分生成的文本
"done": false                // 是否完成
}
```

5. 编程语言示例

Python 使用 requests 库与 Ollama API 交互:

实例

```
import requests

# 生成文本
response = requests.post(
    "http://localhost:11434/api/generate",
    json={
        "model": "deepseek-coder",
        "prompt": "你好, 你能帮我写一段代码吗? ",
        "stream": False
    }
)
print(response.json())
```

多轮对话:

实例

```
response = requests.post(
    "http://localhost:11434/api/chat",
    json={
        "model": "deepseek-coder",
        "messages": [
            {
                "role": "user",
                "content": "你好, 你能帮我写一段 Python 代码吗? "
            }
        ],
        "stream": False
    }
)
print(response.json())
```

JavaScript 使用 fetch API 与 Ollama 交互:

实例

// 生成文本

```
fetch("http://localhost:11434/api/generate", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({
    model: "deepseek-coder",
    prompt: "你好，你能帮我写一段代码吗？",
    stream: false
  })
})
.then(response => response.json())
.then(data => console.log(data));
```

多轮对话:

实例

```
fetch("http://localhost:11434/api/chat", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({
    model: "deepseek-coder",
    messages: [
      {
        role: "user",
        content: "你好，你能帮我写一段 Python 代码吗？"
      }
    ],
    stream: false
  })
})
.then(response => response.json())
.then(data => console.log(data));
```

Ollama Python 使用

Ollama 提供了 Python SDK，可以让我们能够在 Python 环境中与本地运行的模型进行交互。通过 Ollama 的 Python SDK 能够轻松地将自然语言处理任务集成到 Python 项目中，执行各种操作，如文本生成、对话生成、模型管理等，且不需要手动调用命令行。

安装 Python SDK

首先，我们需要安装 Ollama 的 Python SDK。

可以使用 pip 安装：

```
pip install ollama
```

确保你的环境中已安装了 Python 3.x，并且网络环境能够访问 Ollama 本地服务。

启动本地服务

在使用 Python SDK 之前，确保 Ollama 本地服务已经启动。

你可以使用命令行工具来启动它：

```
ollama serve
```

启动本地服务后，Python SDK 会与本地服务进行通信，执行模型推理等任务。

使用 Ollama 的 Python SDK 进行推理

安装了 SDK 并启动了本地服务后，我们就可以通过 Python 代码与 Ollama 进行交互。

首先，从 ollama 库中导入 chat 和 ChatResponse：

```
from ollama import chat
from ollama import ChatResponse
```

通过 Python SDK，你可以向指定的模型发送请求，生成文本或对话：

实例

```
from ollama import chat
from ollama import ChatResponse

response: ChatResponse = chat(model='deepseek-coder', messages=[
    {
        'role': 'user',
```

```
    'content': '你是谁? ',
},
])
# 打印响应内容
print(response['message']['content'])

# 或者直接访问响应对象的字段
#print(response.message.content)
```

执行以上代码，输出结果为：

我是由中国的深度求索（DeepSeek）公司开发的编程智能助手，名为 DeepCoder。我可以帮你解答与计算机科学相关的问题和任务。如果你有任何关于这方面的话题或者需要在某个领域进行学习或查询信息时请随时提问！

llama SDK 还支持流式响应，我们可以在发送请求时通过设置 `stream=True` 来启用响应流式传输。

实例

```
from ollama import chat

stream = chat(
    model='deepseek-coder',
    messages=[{'role': 'user', 'content': '你是谁?'}],
    stream=True,
)

# 逐块打印响应内容
for chunk in stream:
    print(chunk['message']['content'], end='', flush=True)
```

自定义客户端

你还可以创建自定义客户端，来进一步控制请求配置，比如设置自定义的 `headers` 或指定本地服务的 `URL`。

创建自定义客户端

通过 `Client`，你可以自定义请求的设置（如请求头、`URL` 等），并发送请求。

实例

```

from ollama import Client

client = Client(
    host='http://localhost:11434',
    headers={'x-some-header': 'some-value'}
)

response = client.chat(model='deepseek-coder', messages=[
    {
        'role': 'user',
        'content': '你是谁?',
    },
])
print(response['message']['content'])

```

异步客户端

如果你希望异步执行请求，可以使用 `AsyncClient` 类，适用于需要并发的场景。

实例

```

import asyncio
from ollama import AsyncClient

async def chat():
    message = {'role': 'user', 'content': '你是谁?'}
    response = await AsyncClient().chat(model='deepseek-coder', messages=[message])
    print(response['message']['content'])

asyncio.run(chat())

```

异步客户端支持与传统的同步请求一样的功能，唯一的区别是请求是异步执行的，可以提高性能，尤其是在高并发场景下。

异步流式响应

如果你需要异步地处理流式响应，可以通过将 `stream=True` 设置为异步生成器来实现。

实例

```
import asyncio
from ollama import AsyncClient

async def chat():
    message = {'role': 'user', 'content': '你是谁?'}
    async for part in await AsyncClient().chat(model='deepseek-coder', messages=[message], stream=True):
        print(part['message']['content'], end='', flush=True)

asyncio.run(chat())
```

这里，响应将逐部分地异步返回，每部分都可以即时处理。

常用 API 方法

Ollama Python SDK 提供了一些常用的 API 方法，用于操作和管理模型。

1. chat 方法

与模型进行对话生成，发送用户消息并获取模型响应：

```
ollama.chat(model='llama3.2', messages=[{'role': 'user', 'content': 'Why is the sky blue?'}])
```

2. generate 方法

用于文本生成任务。与 chat 方法类似，但是它只需要一个 prompt 参数：

```
ollama.generate(model='llama3.2', prompt='Why is the sky blue?')
```

3. list 方法

列出所有可用的模型：

```
ollama.list()
```

4. show 方法

显示指定模型的详细信息：

```
ollama.show('llama3.2')
```

5. create 方法

从现有模型创建新的模型：

```
ollama.create(model='example', from_='llama3.2', system="You are Mario from Super Mario Bros.")
```

6. copy 方法

复制模型到另一个位置：

```
ollama.copy('llama3.2', 'user/llama3.2')
```

7. delete 方法

删除指定模型：

```
ollama.delete('llama3.2')
```

8. pull 方法

从远程仓库拉取模型：

```
ollama.pull('llama3.2')
```

9. push 方法

将本地模型推送到远程仓库：

```
ollama.push('user/llama3.2')
```

10. embed 方法

生成文本嵌入：

```
ollama.embed(model='llama3.2', input='The sky is blue because of rayleigh scattering')
```

11. ps 方法

查看正在运行的模型列表：

```
ollama.ps()
```

错误处理

Ollama SDK 会在请求失败或响应流式传输出现问题时抛出错误。

我们可以使用 `try-except` 语句来捕获这些错误，并根据需要进行处理。

实例

```
model = 'does-not-yet-exist'
```

```
try:
```

```
    response = ollama.chat(model)
```

```
except ollama.ResponseError as e:
```

```
    print('Error:', e.error)
```

```
    if e.status_code == 404:
```

```
        ollama.pull(model)
```

在上述例子中，如果模型 `does-not-yet-exist` 不存在，抛出 `ResponseError` 错误，捕获后你可以选择拉取该模型或进行其他处理。

Ollama Open WebUI

Open WebUI 用户友好的 AI 界面（支持 Ollama、OpenAI API 等）。

Open WebUI 支持多种语言模型运行器（如 Ollama 和 OpenAI 兼容 API），并内置了用于检索增强生成（RAG）的推理引擎，使其成为强大的 AI 部署解决方案。

Open WebUI 可自定义 OpenAI API URL，连接 LMStudio、GroqCloud、Mistral、OpenRouter 等。

Open WebUI 管理员可创建详细的用户角色和权限，确保安全的用户环境，同时提供定制化的用户体验。

Open WebUI 支持桌面、笔记本电脑和移动设备，并提供移动设备上的渐进式 Web 应用（PWA），支持离线访问。

开源地址: <https://github.com/open-webui/open-webui>

官方文档: <https://docs.openwebui.com/>

安装

Open WebUI 提供多种安装方式，包括通过 Python pip 安装、Docker 安装、Docker Compose、Kustomize 和 Helm 等。

使用 Docker 快速开始

如果 Ollama 已安装在你的电脑上，使用以下命令：

```
docker run -d -p 3000:8080 --add-host=host.docker.internal:host-gateway -v
open-webui:/app/backend/data --name open-webui --restart always
ghcr.io/open-webui/open-webui:main
```

使用 Nvidia GPU 支持运行 Open WebUI:

```
docker run -d -p 3000:8080 --gpus all --add-host=host.docker.internal:host-gateway
-v open-webui:/app/backend/data --name open-webui --restart always
ghcr.io/open-webui/open-webui:cuda
```


Open WebUI 与 Ollama 捆绑安装

此安装方法使用一个单一的容器镜像，将 Open WebUI 与 Ollama 捆绑在一起，可以通过一个命令轻松设置。

根据你的硬件配置选择合适的命令。

启用 GPU 支持：

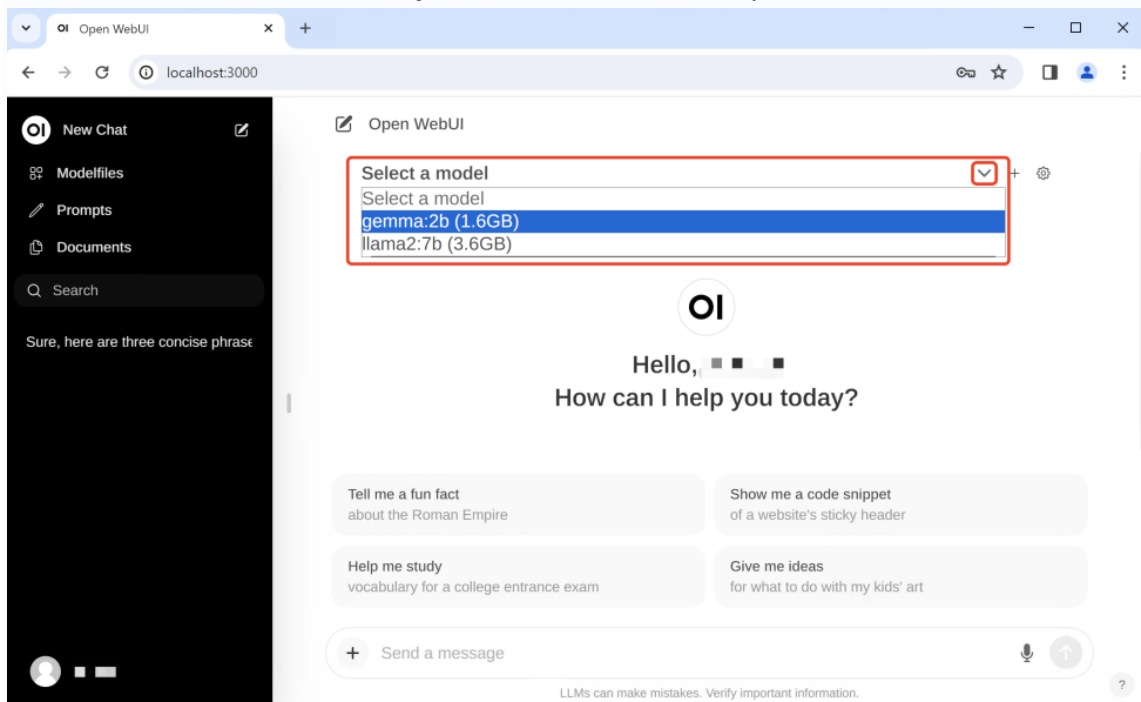
```
docker run -d -p 3000:8080 --gpus=all -v ollama:/root/.ollama -v open-webui:/app/backend/data --name open-webui --restart always ghcr.io/open-webui/open-webui:ollama
```

仅使用 CPU：

```
docker run -d -p 3000:8080 -v ollama:/root/.ollama -v open-webui:/app/backend/data --name open-webui --restart always ghcr.io/open-webui/open-webui:ollama
```

这两个命令都能帮助你简化安装过程，让 Open WebUI 和 Ollama 无缝运行。

安装完成后，你可以通过访问 <http://localhost:3000> 使用 Open WebUI。



更新 Open WebUI

手动更新

使用 Watchtower 手动更新 Docker 容器：

```
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock containrrr/watchtower
--run-once open-webui
```

自动更新

每 5 分钟自动更新容器：

```
docker run -d --name watchtower --restart unless-stopped -v
/var/run/docker.sock:/var/run/docker.sock containrrr/watchtower --interval 300
open-webui
```

注意：如果你的容器名不同，请将 `open-webui` 替换为你的容器名。

手动安装

Open WebUI 的安装有两种主要方法：使用 `uv runtime manager` 或 `Python` 的 `pip`。

推荐使用 `uv`，因为它简化了环境管理，减少了潜在的冲突。

使用 `uv` 安装（推荐）

macOS/Linux:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Windows:

```
powershell -ExecutionPolicy Bypass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

运行 Open WebUI

安装 `UV` 后，运行 `Open WebUI` 就比较简单了。

使用以下命令，确保设置 `DATA_DIR` 环境变量，以避免数据丢失。

macOS/Linux:

```
DATA_DIR=~/.open-webui uvx --python 3.11 open-webui@latest serve
```

Windows:

```
$env:DATA_DIR="C:\open-webui\data"; uvx --python 3.11 open-webui@latest serve
```

通过 `pip` 安装

`Open WebUI` 可以通过 `Python` 的包安装程序 `pip` 进行安装，在开始安装之前，请确保您使用的是 `Python 3.11`，以避免可能出现的兼容性问题。

打开您的终端，运行以下命令以安装 `Open WebUI`：

```
pip install open-webui
```

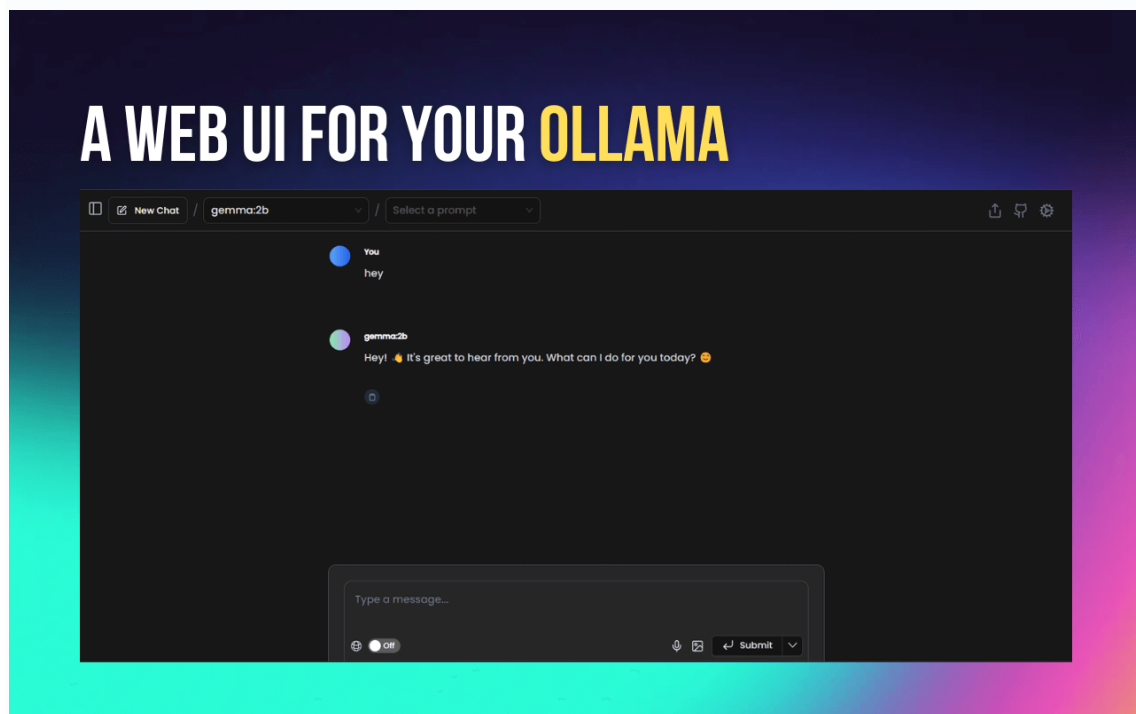
安装完成后，您可以通过以下命令启动 Open WebUI:

```
open-webui serve
```

启动后，Open WebUI 服务器将运行在 <http://localhost:8080>，您可以通过浏览器访问该地址来使用 Open WebUI。

Ollama Page Assist

Page Assist 是一款开源的浏览器扩展程序，主要为本地 AI 模型提供直观的交互界面，让用户可以在任何网页上与本地 AI 模型进行对话和交互。



基本功能

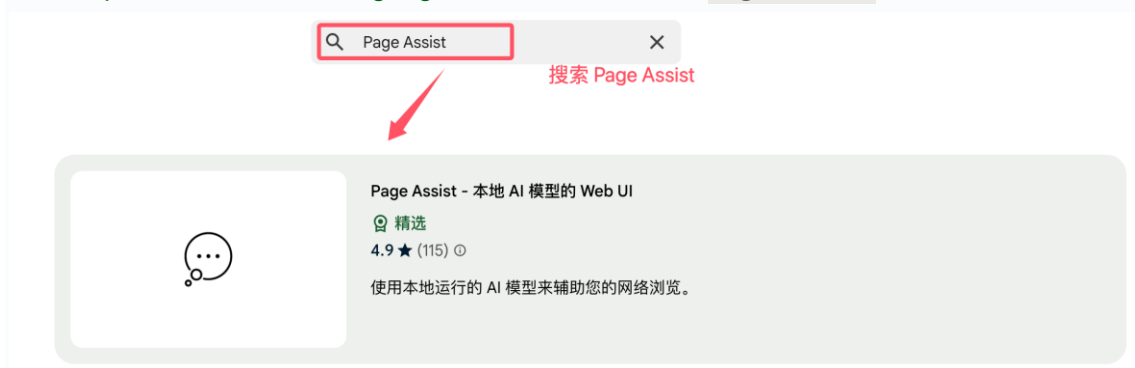
- 侧边栏交互：用户可以在任何网页上打开侧边栏，与本地 AI 模型进行对话，获取与网页内容相关的智能辅助。
- 网页 UI：提供类似 ChatGPT 的网页界面，用户可以在此界面中与 AI 模型进行更全面的对话。
- 网页内容对话：用户可以直接与网页内容进行对话，获取相关信息的解释或分析。
- 多浏览器支持：支持 Chrome、Brave、Edge 和 Firefox 等主流浏览器。
- 快捷键操作：通过快捷键可以快速打开侧边栏和网页 UI，方便用户使用。
- 支持多种本地 AI 提供商：目前支持 Ollama 和 Chrome AI (Gemini Nano) 等本地 AI 提供商。
- 文档解析：支持与 PDF、CSV 等多格式文档进行聊天交流。

- 离线模型适配：适用于离线环境，用户可以在本地运行 AI 模型。
 - 隐私保护：所有交互都在本地完成，不会收集用户个人数据，数据存储浏览器的本地存储中。
- 开发模式：支持开发者模式，便于进行扩展的开发和测试。

安装与使用

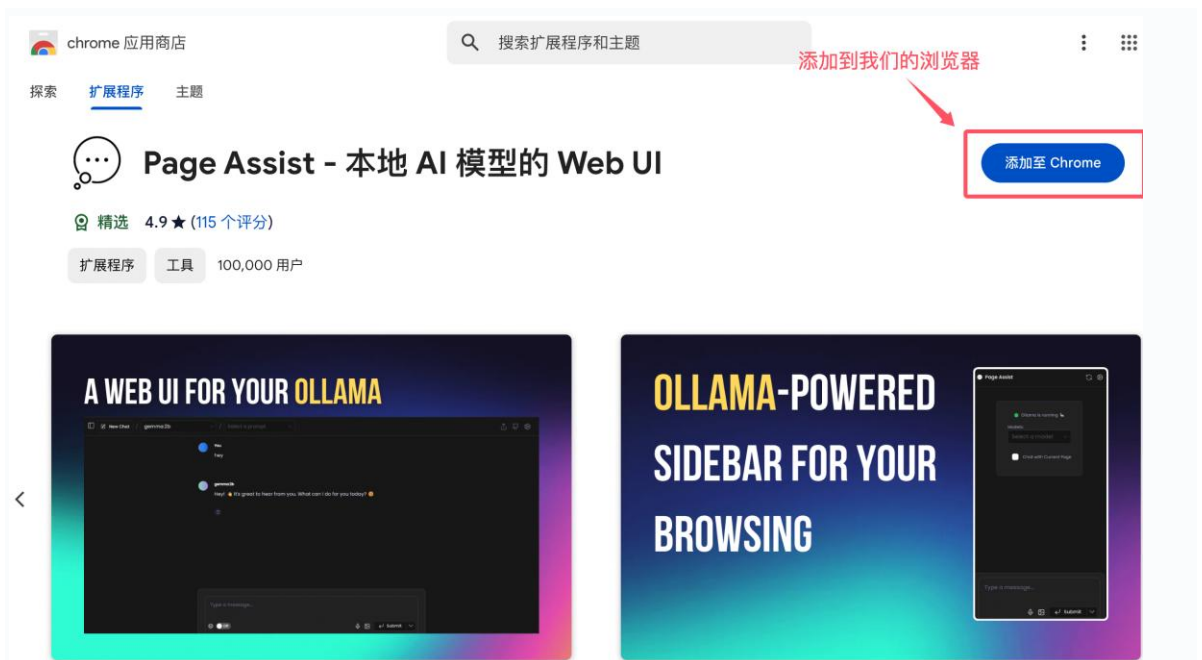
我们可以直接从 Chrome Web Store 或 Firefox Add-ons 商店下载安装，也可以通过手动安装的方式进行。

访问 <https://chromewebstore.google.com/search/>，搜索 **Page Assist**：

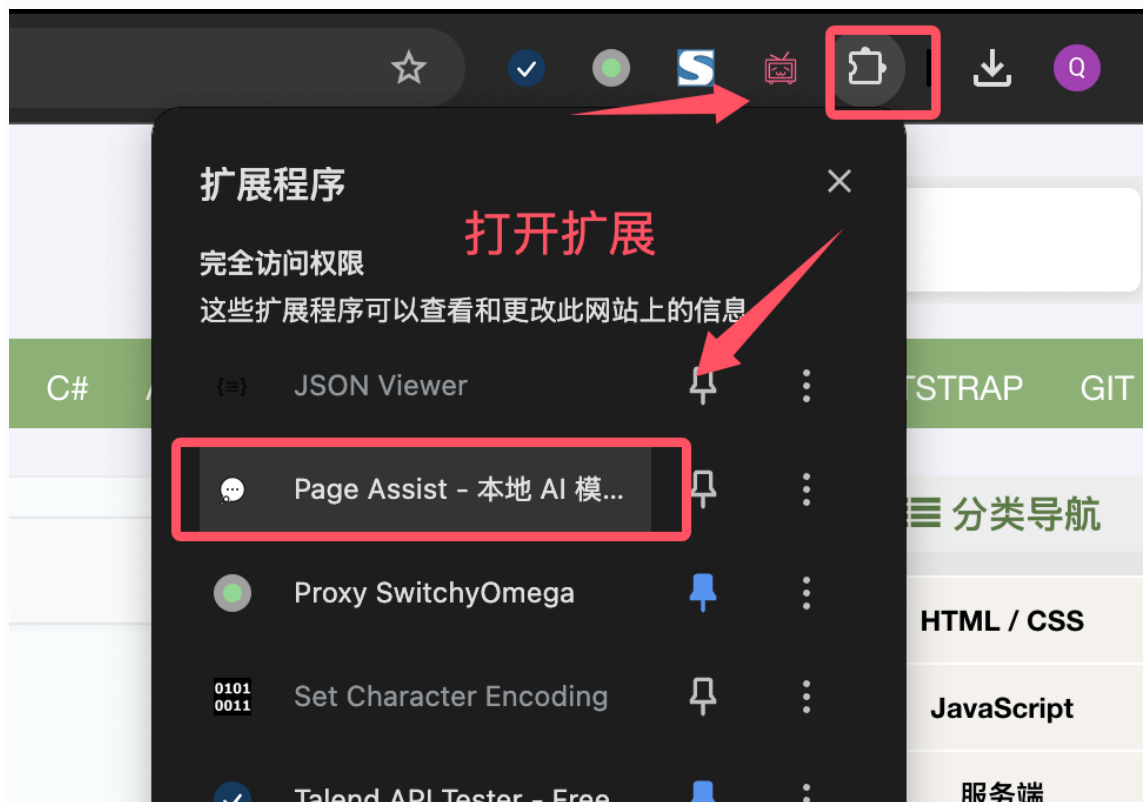


也可以直接点击扩展链接：<https://chromewebstore.google.com/detail/page-assist-%E6%9C%A%E5%9C%B0-ai-%E6%A8%A1%E5%9E%8B%E7%9A%84-web/jfgfiigpkhlkbnfnbobbkinehfhf>dhndo。

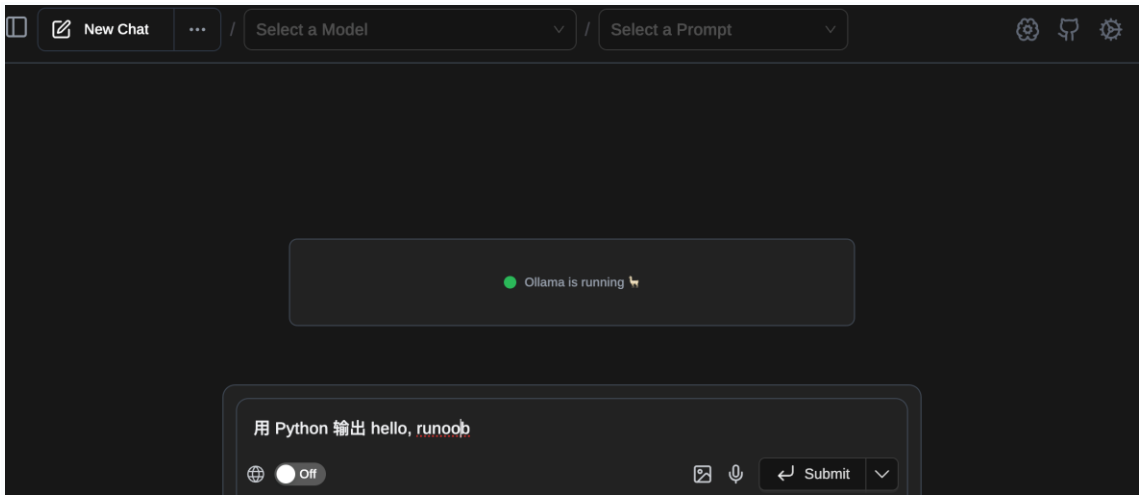
点击添加按钮：



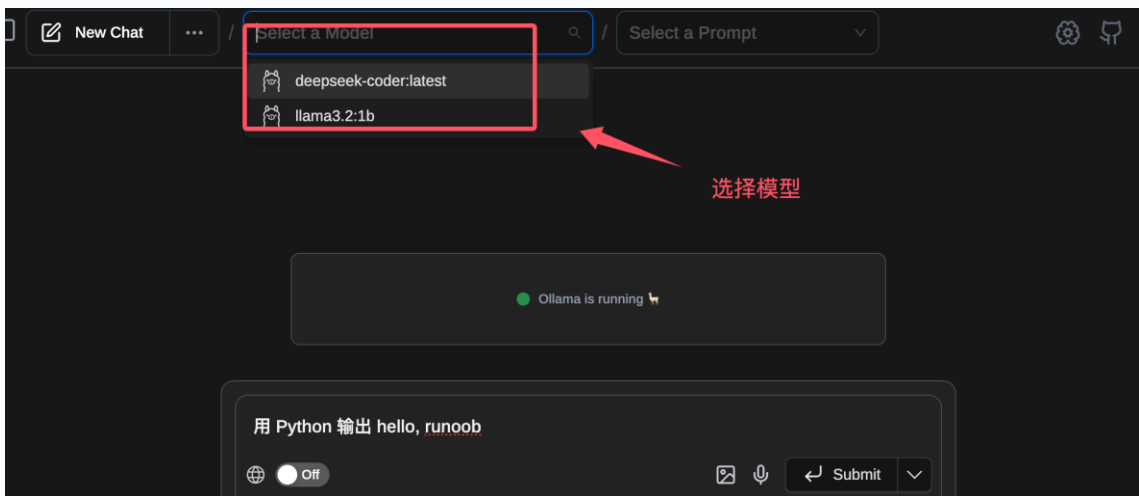
安装完成后，用户可以通过右上角的扩展小标志打开它：



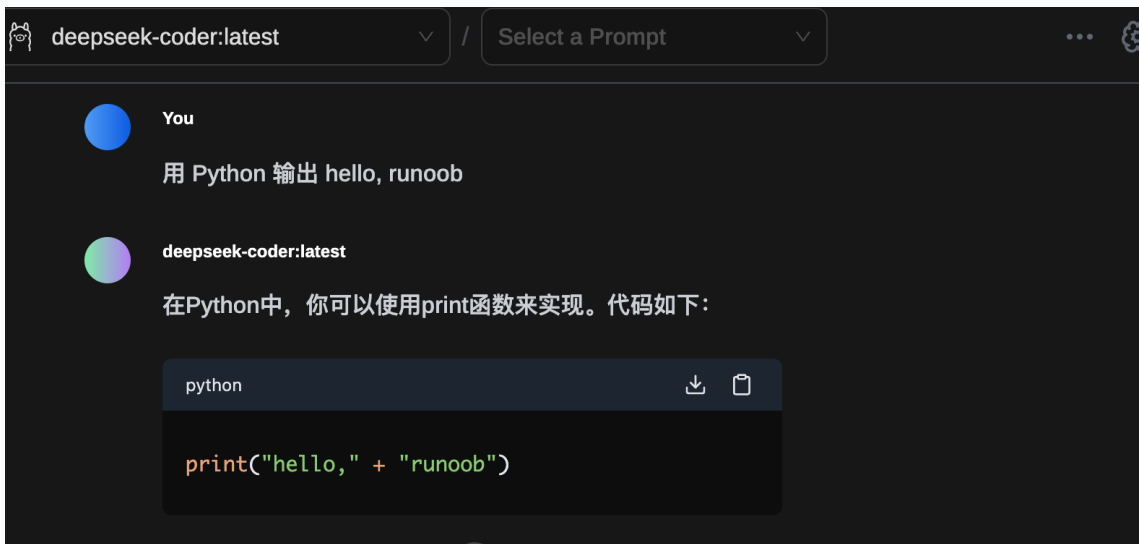
打开后就可以看到 Ollama 正在运行的提示，如果还没启动则需要启动 Ollama：



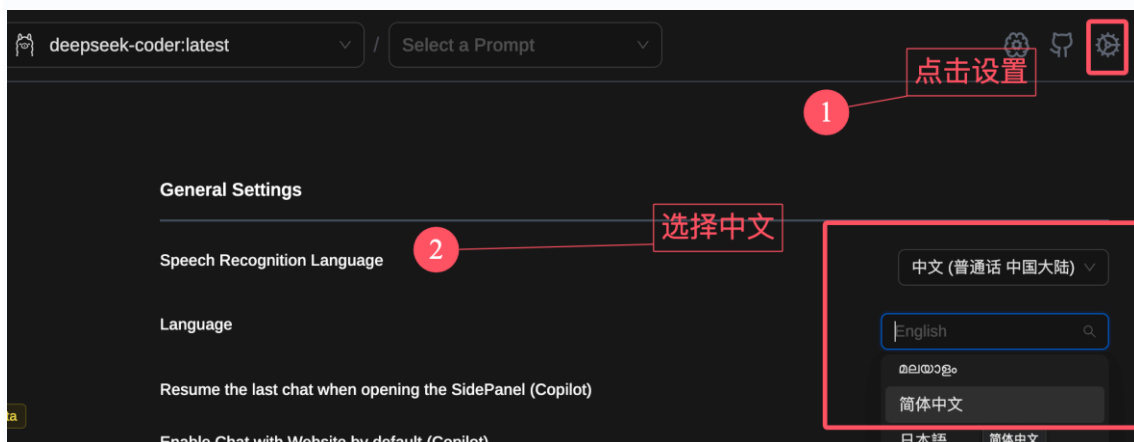
接下来我们可以选择指定的模型，然后输入提示词：



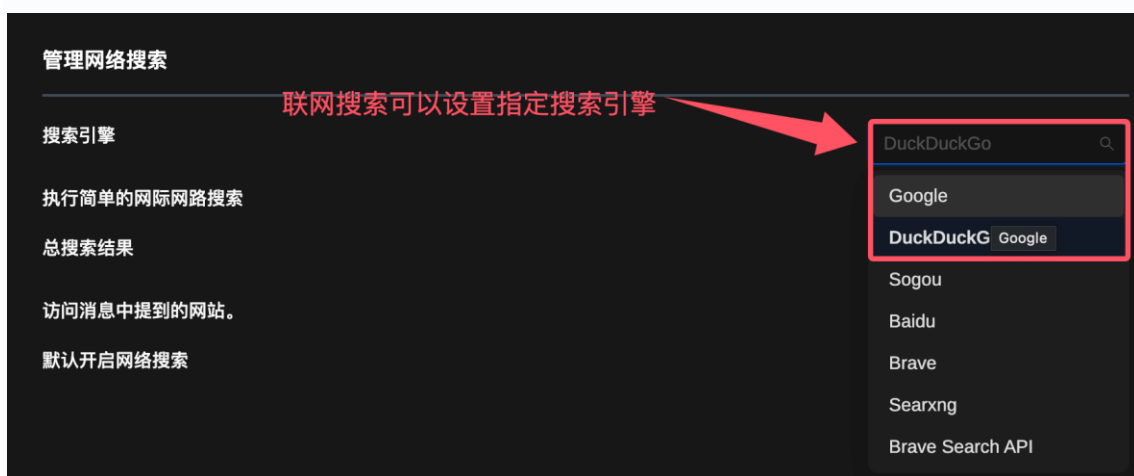
效果如下所示：



界面是英文看不懂，没事，点击右上角的齿轮，可以设置成中文：



联网搜索功能，也可以修改默认搜索引擎：



Page AssistGithub 源代码: <https://github.com/n4ze3m/page-assist>